



VIT[®]
BHOPAL

Project Report : Elder Health Monitoring & SOS Alert System

Institution: VIT Bhopal University

Academic Year: 2026–2027

Guidelines Reference: VITyarthi

Student Name: Atharva Sharma

Registration no : 26BAI10423

Faculty Name : Dr.Jitendra Pratap Singh Mathur

Slot: A21+D21+A22+D22+A23+B14

Introduction

Elderly individuals who live alone often face sudden health risks, such as unexpected raise in body temperature, irregular heartbeats, or severe drops in oxygen levels. Because health conditions can deteriorate quickly, relying on occasional manual check-ups isn't always enough to catch emergencies in time.

The Elder Health Monitoring & SOS Alert System is a modular Python program designed to solve this problem by continuously tracking three main health vitals: Heart Rate (in BPM), Body Temperature (in °C), and Blood Oxygen Level (SpO2 %). The program evaluates incoming health readings against safe medical limits and assigns each reading a status of Normal, Warning, or Critical. If critical conditions are detected, the system immediately triggers a simulated emergency SOS alert to contact caregivers and ambulance services while logging all data into a CSV file for future reference.

Problem Statement & Scope

Senior citizens living independently may experience sudden medical emergencies—such as cardiac fluctuations, severe fevers, or sudden oxygen loss—when no one is around to help. Checking vitals manually a few times a day is slow, easy to forget, and incapable of providing instant emergency alerts when something goes wrong during off-hours.

Target Users

- **Elderly Individuals:** Senior citizens who need an easy way to monitor their health and call for help in an emergency.
- **Family Caregivers:** Family members who want real-time updates and instant notifications if a emergency happens.
- **Doctors & Healthcare Workers:** Medical professionals who need a recorded log of past vital readings to analyze long-term health trends.

Scope

- Simulates live health readings across four distinct modes: Normal, Random, Danger, and Edge-case.

- Automatically categorizes health readings into *Normal*, *Warning*, or *Critical* based on preset thresholds.
- Triggers a simulated SOS sequence to alert emergency services and designated caregivers during critical events.
- Saves all recorded vitals with clear time stamps in a local `vitals_log.csv` file.
- Provides an interactive command-line menu where users can manually enter vitals or trigger an emergency SOS directly.
-

Functional Requirements

In alignment with the VITyarthi project structure guidelines, the system is built around five core functional features(differnent):

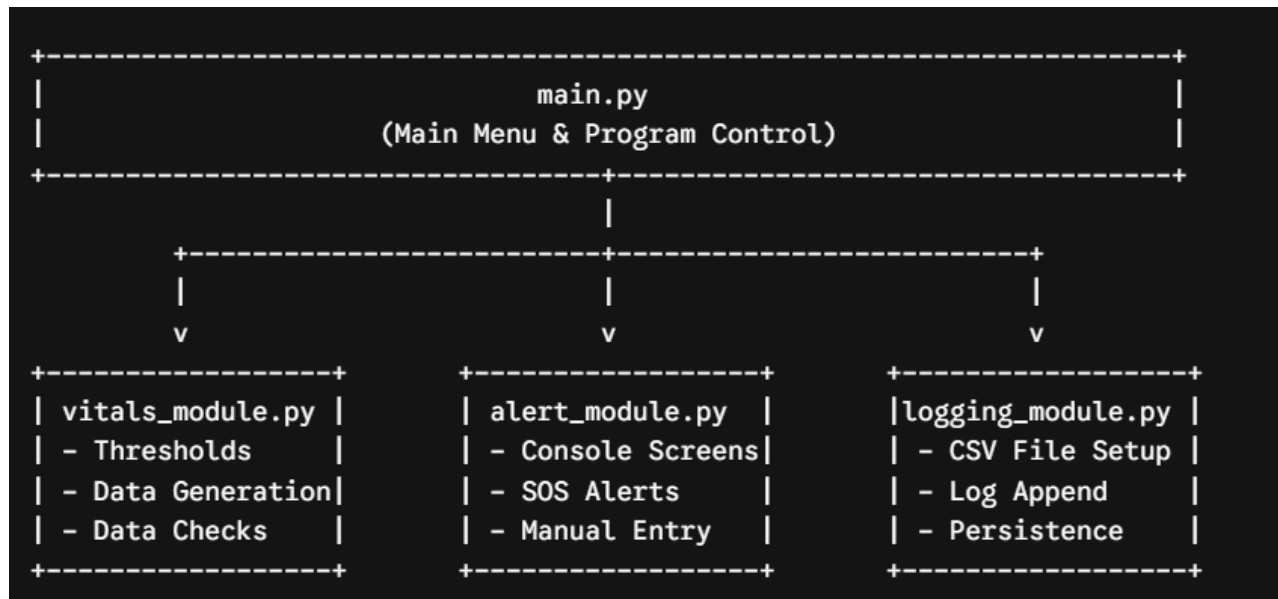
1. Vital Simulation Engine: Generates realistic, random, extreme, or edge-case vital data across different operation modes.
2. Health Check Logic: Compares real-time vitals against preset health limits to catch abnormal values.
3. Emergency SOS Subsystem: Displays an emergency alert screen and simulates calls to ambulances and caregivers during critical situations.
4. CSV Data Loggr: Creates `vitals_log.csv` automatically if it doesn't exist and appends time-stamped health records.
5. Console User Interface: Provides a simple menu for switching modes, typing in custom values, or forcing an SOS alert manually.

Non-Functional Requirements

1. Performance: Processes incoming vital data, checks thresholds, and appends logs in less than 0.1 seconds per cycle.
2. Usability: Features a straightforward text menu that guides the user clearly through all available features.
3. Reliability & Input Safety: Prevents program crashes when invalid data is typed into manual entry modes by using try-except validation blocks.

4. Data Persistence: Keeps a permanent record of all readings in a local CSV file so no data is lost when the program closes.
5. Maintainability: Written using a clean, modular structure split across four Python files (main.py, vitals_module.py, alert_module.py, and logging_module.py).

System Architecture

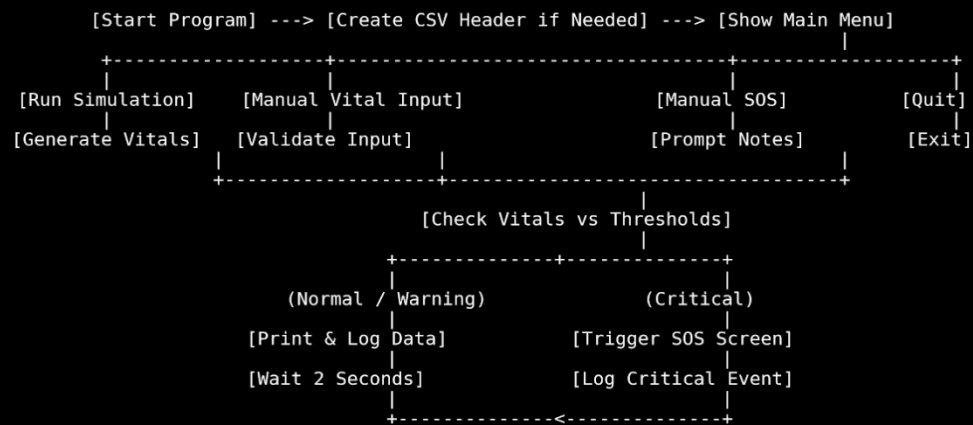


Design Diagrams

7.1 Use Case Diagram

- Actors: Patient/Elderly User, Caregiver/Admin.
- Use Cases: Choose Simulation Mode, Enter Vitals Manually, Trigger Manual SOS, View Live Health Status, Log Data to CSV, Dispatch Emergency SOS.

7.2 Workflow Diagram



7.3 Sequence Diagram

1. The user selects an option from the menu in main.py.
2. main.py calls `vitals_module.py` to generate or receive readings.
3. `vitals_module.py` checks the values against preset thresholds and returns a status (*Normal*, *Warning*, or *Critical*) along with a descriptive note.
4. main.py calls `alert_module.py` to format and display the status on the console.
5. main.py calls `logging_module.py` to save the reading into `vitals_log.csv`.
6. If the reading is *Critical*, main.py calls `trigger_sos()` in `alert_module.py` to trigger the emergency screen.

7.4 Class / Component Diagram

- `main.py`: Controls the main event loop, handles user selection, and directs traffic between modules.
- `vitals_module.py`: Stores the `THRESHOLDS` dictionary and handles `simulate_vitals()` and `check_vitals()` functions.

- `alert_module.py`: Stores emergency contact details (EMERGENCY_CONTACT) and manages menu printing, manual inputs, and SOS displays.
- `logging_module.py`: Manages file paths, creates CSV files (`ensure_csv_header()`), and writes rows (`log_reading()`).

Design Decisions & Rationale

- Modular Python Architecture: Splitting functionality across `main.py`, `vitals_module.py`, `alert_module.py`, and `logging_module.py` keeps the project organized, making it much easier to debug and test individual parts.
- CSV File Storage: Using standard CSV files through Python's built-in `csv` library removes the need to install external database software while keeping the logs easy to open in spreadsheet programs like Excel.
- Centralized Threshold Dictionary: Storing health boundaries in a single dictionary in `vitals_module.py` allows medical limits to be updated easily without changing the core evaluation logic.
- Text-Based Console UI: A lightweight terminal interface ensures the application runs smoothly on any system without requiring extra GUI libraries.

Implementation Details

Threshold Configuration (`vitals_module.py`)

Normal and critical values used to evaluate readings

```
THRESHOLDS = {
    "heart_rate": {
        "normal_min": 60,
        "normal_max": 100,
        "warning_max": 120,
        "critical_low": 40,
        "critical_high": 120,
```

```

},
"temperature_c": {
    "normal_min": 36.1,
    "normal_max": 37.2,
    "warning_max": 38.9,
    "critical_high": 39.0,
},
"spo2": {
    "normal_min": 94,
    "warning_min": 90,
    "critical_min": 90,
},

```

Emergency Alert Logic (alert_module.py)

```
def trigger_sos(hr, temp_c, spo2, note):
```

```

    """Display the SOS alert and simulate contacting emergency services."""
    print("\n" + "!" * 60)

    print(" CRITICAL ALERT - SOS TRIGGERED ")

    print(f"Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

    print(f"Detected critical vitals -> HR: {hr} bpm, Temp: {temp_c}°C, SpO2: {spo2}%")

    print(f>Note: {note}")

    print("Calling emergency services (SIMULATION)...")

    print(f"Ambulance: {EMERGENCY_CONTACT['ambulance_number']}")

    print(f"Caregiver: {EMERGENCY_CONTACT['caregiver_name']}")
    ({EMERGENCY_CONTACT['caregiver_number']})

    print("!" * 60 + "\n")

```

```
# Store the SOS event in the CSV file
```

```
log_reading(CSV_FILENAME, hr, temp_c, spo2, "CRITICAL", note + " | SOS triggered")
```

Sample Output (Danger Simulation Mode)

[illegible]

[INFO] Returning to menu...

Testing Approach

TEST CASE ID	TEST SCENARIO	INPUT DATA	EXPECTED RESULT	STATUS
TC-01	Normal Mode Simulation	HR: 72, Temp: 36.6 °C, SpO2: 98%	Status is Normal; logged to CSV	PASS
TC-02	Danger Mode Simulation	HR: 135, Temp: 39.4 °C, SpO2: 86%	Status is Critical; triggers SOS log	PASS
TC-03	Edge-Case Testing	HR: 120, Temp: 38.9 °C, SpO2: 90%	Status is Warning; no SOS triggered	PASS
TC-04	Invalid Manual Input Handling	Inputting "abc" for Heart Rate	Catches error; prompts user to re-enter	PASS
TC-05	Manual SOS Command	Selecting option "s" from main menu	Triggers emergency SOS screen immediately	PASS

Challenges Faced

1. **Handling Edge Cases:** Determining exactly when a reading switches from *Warning* to *Critical* required writing precise if-elif statements so boundary values (like 38.9°C vs 39.0°C) trigger the correct status.
2. **Graceful User Exit:** Stopping active simulation loops cleanly using Ctrl+C was tricky at first. Adding a KeyboardInterrupt exception block fixed this so users return smoothly to the main menu without seeing long error tracebacks.
3. **Preventing Input Crashes:** In manual mode, typing letters or leaving fields empty caused program crashes. Wrapped the input parsers in try-except blocks to handle bad inputs safely.

Learnings & Key Takeaways

- Gained hands-on experience structuring a Python project using multiple separate modules.
- Learned how to read, write, and safely append data to CSV files using standard file I/O operations.
- Practiced handling user errors and control flow with try-except blocks and loop controls.

Future Enhancements

1. Hardware Integration: Connect real physical sensors (such as MAX30102 pulse oximeter and DS18B20 temperature sensors) via an ESP32 or Raspberry Pi board.
2. Real SMS & Call Alerts: Use messaging services like Twilio or WhatsApp APIs to send actual SMS texts and automated phone calls to family members when an SOS is triggered.
3. GraphicalUser Interface (GUI): Built a visual desktop interface using Tkinter or CustomTkinter to show live graphs of heart rate and oxygen levels over time.

References

1. World Health Organization (WHO). *Pulse Oximetry Training Manual & Vital Signs Reference Limits*.
2. VIT Bhopal University. *Build Your Own Project Guidelines – VITyarthi Platform* and with vityarthi flipped course And also in guidance with our course respected faculty Dr. Jitendra Pratap Singh Mathur.

